

# cppGFS2.0 — System Design (2026 Edition)

Architecture reference for the current implementation — August 2026. Companion to the original CS244B course paper.

This document describes the architecture of cppGFS2.0 as it stands today. It updates and supersedes the system description in the original course paper (*Google File System 2.0: A Modern Design and Implementation*, CS244B, 2020 — kept unchanged in `data/CS244B_Final_Paper.pdf` as the historical record). Since that paper was written, the implementation has been brought substantially closer to the design in the original GFS paper (Ghemawat, Gobioff, and Leung, *The Google File System*, SOSP 2003); section references below (e.g. §4.5) refer to that paper.

## 1. Overview

A cppGFS2.0 cluster consists of a single **master**, a configurable number of **chunk servers**, and a **client library**. Files are divided into fixed-size chunks (64MB by default), identified by immutable, globally unique chunk handles assigned by the master at creation time. Each chunk is stored on multiple chunk servers (3 by default, configurable via `disk.replication_factor`).

Clients contact the master only for metadata (which chunk handle backs a given file offset, and which chunk servers hold it); all file data flows directly between clients and chunk servers. All servers communicate over gRPC, and cluster topology, chunk size, replication factor, storage paths, and timeouts are configured in a single YAML file (`data/config.yml`).

## 2. The master

### 2.1 Metadata and persistence

The master keeps three kinds of metadata in memory:

- the file namespace: a map from pathname to file metadata, including the file's chunk-index-to-chunk-handle mapping;
- per-chunk metadata: the chunk's current version number;
- runtime state: the chunk servers holding each chunk, and lease state.

Following §2.6 of the GFS paper, the first two are persistent and the third is not. When a LevelDB path is configured for the master (`disk.leveldb.<master_name>`), every namespace or chunk-version mutation is synchronously written through to the on-disk metadata store *before* the mutation is acknowledged to the client — the same durability contract as the paper's operation log ("respond to a client operation only after flushing the corresponding log record"), with the store playing the combined role of log and checkpoint. On startup the master recovers the namespace, chunk versions, and the chunk-handle allocator from this store, so files survive a master crash or restart. Chunk *locations* are deliberately not persisted: the master re-learns them from chunk server reports (§2.6.2), which arrive on startup and periodically thereafter.

### 2.2 Namespace locking

The namespace is a flat map from full pathnames to metadata, with a read-write lock per path (§4.1). Operations take read locks on all ancestor paths and a read or write lock on the leaf, which serializes conflicting operations

(e.g. two creations of the same file) while allowing concurrent mutations in the same directory.

## 2.3 Chunk server management

Chunk servers report themselves to the master on startup and periodically (by default every `timeout.heartbeat`). A report carries the server's available disk and the handle *and version* of every chunk it stores. The master uses reports to (re-)register servers, to learn chunk locations, and to detect stale replicas (see §5.2 below). For chunk allocation the master prefers servers with the most available disk, which spreads load and equalizes utilization (§4.3's "below-average disk space utilization" criterion, simplified).

A background heartbeat task pings every registered chunk server; a server that fails several consecutive heartbeats is unregistered, immediately removing it from every chunk's location set so clients are no longer directed to it. The same background pass runs a replication repair scan (§2.6.1, §4.3): any chunk whose live replica count is below the replication goal is re-replicated by instructing a chunk server that does not yet hold the chunk to clone it directly from an existing valid replica (a chunk-server-to-chunk-server copy; the data does not pass through the master). The scan runs every heartbeat period, so repairs also happen when a replacement server joins the cluster later.

## 3. Mutations: leases, versions, and mutation order

Writes follow the control flow of §3.1 of the GFS paper:

- 1 The client asks the master (an `OpenFile` request in `WRITE` mode) which chunk server holds the lease for the chunk and where the replicas are.
- 2 If a still-valid lease exists and its holder is still live, the master returns the existing primary, the current chunk version, and the replica locations — the version does **not** advance. If no valid lease exists, the master grants a new one: it increments the chunk version, informs all up-to-date replicas (each replica accepts the advance only if it is exactly one version behind), records the new version persistently, and then grants the lease to one accepting replica, which becomes the **primary** (§4.5: "Whenever the master grants a new lease on a chunk, it increases the chunk version number and informs the up-to-date replicas ... before any client is notified"). Replicas that fail to advance are excluded from the locations returned to the client. The whole sequence is serialized per chunk, so concurrent write-opens are idempotent: both clients receive the same primary and version.
- 3 The client pushes the data to all replicas, identified by checksum. Each chunk server stages the data in an in-memory cache. (This is the one deliberate simplification of §3.2: data is pushed from the client to all replicas in parallel rather than pipelined along a chunk server chain.)
- 4 The client sends the write request to the primary. The primary validates its lease, then applies the mutation locally and forwards it to each secondary replica. A per-chunk mutation lock is held across the local apply and the forwarding, so concurrent writes to the same chunk are applied in one serial order — the lock acquisition order at the primary is the mutation order, and every secondary receives mutations one at a time in that order. This provides §3.1's guarantee that all replicas apply mutations identically.
- 5 The primary reports each secondary's outcome to the client. The client treats a failure at **any** replica as a failed write (§3.1 step 7), refreshes its metadata, re-pushes the data, and retries; a lease change discovered during retry (`FAILED_NOT_LEASE_HOLDER`) likewise triggers a metadata refresh.

Leases expire after `timeout.lease` (60s by default) and the master may revoke them. Lease state is soft: after a master restart, new leases are granted afresh, which is safe because granting a new lease advances the version

on the up-to-date replicas.

## 4. Reads

The client asks the master for the chunk handle, version, and locations (caching them with a configurable TTL), then reads from any replica, supplying the version. A replica rejects reads for a version it does not have, and reads whose stored data fails checksum verification, so a client never observes stale or corrupt data from a single bad replica — it simply fails over to the next one. If every cached location fails (stale cache after another client's write, or servers went away), the client refreshes its metadata from the master and retries once (§2.7.1: "When a reader retries and contacts the master, it will immediately get current chunk locations").

## 5. Fault tolerance

### 5.1 Data integrity (§5.2)

Each chunk server independently verifies the integrity of its own data with block checksums: a 32-bit CRC (CRC32C) per 64KB block of chunk data, stored alongside the chunk. Checksums are verified on every read that overlaps a block before any data is returned, so corruption is never propagated to clients or to other chunk servers (clone reads use the same verified path). On writes, checksums are updated incrementally for the touched blocks; for a partial overwrite, the first and last overlapped blocks are verified *first*, so a new checksum can never hide pre-existing corruption in the region not being overwritten. A corrupt replica surfaces as a distinct error status; the client fails over to another replica, and the replica can be restored by the re-replication scan.

### 5.2 Stale replica detection (§4.5)

A replica becomes stale when its chunk server misses mutations while down. Because the version advances on every new lease grant, staleness is detectable by version comparison: when a chunk server reports its chunks, the master compares each reported version with its own record. A replica *behind* the master's version is stale — it is dropped from the chunk's location set and the chunk server is told to delete it in the report reply. A replica *ahead* of the master's record means the master failed after instructing replicas to advance but before persisting the new version; the master adopts the higher version as up to date, exactly as prescribed by §4.5.

### 5.3 Garbage collection (§4.4)

Deleting a file removes its namespace entry and chunk metadata at the master immediately, but the chunk data on chunk servers is reclaimed lazily through the regular report exchange: when a chunk server reports a chunk the master no longer knows, the reply lists it as garbage and the chunk server deletes its replica. This mirrors the paper's heartbeat-based garbage collection at the chunk level (the paper's additional hidden-rename grace period at the file level is not implemented; deletion of the namespace entry is immediate).

### 5.4 Re-replication (§4.3)

Covered in §2.3 above: the master's background scan detects chunks below the replication goal and directs an available chunk server to clone the chunk from a live replica at the current version. Combined with stale replica deletion, this makes the cluster self-healing: a chunk server that returns after losing or corrupting data has its bad replicas deleted and fresh ones cloned back.

### 5.5 Fast recovery (§5.1.1)

Both server types restore their state and start in seconds. The master replays its metadata store; chunk servers scan their local LevelDB and report inventory to the master. Neither distinguishes normal from abnormal termination.

## 6. Consistency model

The guarantees match Table 1 of the GFS paper for the supported operations: namespace mutations (creation, deletion) are atomic, handled exclusively at the master under namespace locking. A write that succeeds serially is defined; concurrent successful writes to the same region leave it consistent (all replicas identical, thanks to primary-ordered mutations) but possibly undefined (mingled fragments); a failed write leaves the region inconsistent, and clients are expected to retry. Clients may briefly read stale data only through their own metadata cache (bounded by the cache TTL and purged on refresh), never from a stale replica.

## 7. Known deviations from the GFS paper, and future work

- **Data flow:** clients push data to all replicas in parallel instead of pipelining through a chunk server chain (§3.2). This simplifies error handling at some cost in client egress bandwidth.
- **Record append and snapshot** (§3.3, §3.4) are not implemented; writes are at client-specified offsets only.
- **Single master:** there is no operation-log replication to remote machines and no shadow masters (§5.1.3). The persistent metadata store covers master *restarts*, not the loss of the master's disk.
- **File-level deletion grace period** (§4.4.1's hidden rename with lazy reclamation after three days) is not implemented; namespace removal is immediate, chunk reclamation is lazy.
- **Placement is disk-based only:** rack-aware placement and rebalancing (§4.2) are out of scope for a single-machine/containerized deployment.
- **Transport security:** all gRPC channels are unauthenticated and unencrypted, as in the original course project; the system is intended to run on a trusted network or inside a container network. Adding TLS/mTLS via gRPC credentials is mechanical future work.

## 8. Code map

| Component                                            | Location                                                             |
|------------------------------------------------------|----------------------------------------------------------------------|
| Master: metadata + persistence                       | src/server/master_server/metadata_manager.{h,cc}                     |
| Master: namespace locking                            | src/server/master_server/lock_manager.{h,cc}                         |
| Master: leases, versions, write/read/create handling | src/server/master_server/master_metadata_service_impl.cc             |
| Master: chunk server registry & allocation           | src/server/master_server/chunk_server_manager.{h,cc}                 |
| Master: report handling, stale replica detection     | src/server/master_server/master_chunk_server_manager_service_impl.cc |
| Master: heartbeats, re-replication scan              | src/server/master_server/chunk_server_heartbeat_monitor_task.{h,cc}  |
| Chunk server: storage engine + checksums             | src/server/chunk_server/file_chunk_manager.{h,cc}                    |

| Component                                           | Location                                                   |
|-----------------------------------------------------|------------------------------------------------------------|
| Chunk server: read/write/clone RPCs, mutation order | src/server/chunk_server/chunk_server_file_service_impl.cc  |
| Chunk server: leases                                | src/server/chunk_server/chunk_server_lease_service_impl.cc |
| Chunk server: reports, self state                   | src/server/chunk_server/chunk_server_impl.{h,cc}           |
| Client library                                      | src/client/                                                |
| Wire protocol                                       | src/protos/, src/protos/grpc/                              |